Learning Compositional Meta-Routing for Agentic Workflows: An Executable Benchmark

Natan Vidra[1], **Alina Kapanova**[1,2], **Arun Kanhai**[1,3], **Spurthi Setty**[4]
[1]Anote AI
[2]Cornell University
[3]CUNY
[4]Stevens Institute of Technology
nvidra@anote.ai, ak2765@cornell.edu, arun.kanhai55@qmail.cuny.edu, ssetty2@stevens.edu

Abstract

Agentic systems must decide not only what answer to produce, but which reasoning and execution operations should precede it. A controller may answer directly, decompose a request, retrieve evidence, execute code, delegate to a specialist, or verify an intermediate result. Existing routing work largely selects model endpoints, retrieval depth, or tools in isolation. We introduce an executable benchmark and a budget-aware meta-router that composes heterogeneous operations from raw task text. The benchmark contains 216 training, 72 development, 108 held-out test, and 108 locked lexical-shift challenge tasks across data analysis, frozen-corpus research, and document processing. Outcomes are machine checked after operations execute. Independent regularized logistic heads predict operation probabilities from word and character features, are temperature-scaled on development data, and are greedily composed under route-cost and action-count budgets. On the held-out test, the learned policy achieves 100% success versus 93.5% for strong static and fixed workflows, with 43% lower cost than the static policy; a matched learned one-shot router reaches 56.5%. On the untouched challenge split, learned success falls to 75.9% and trails static routing at 93.5%, while remaining 49% cheaper and exceeding one-shot routing by 34.3 points. The gap identifies lexical generalization, rather than route execution, as the principal limitation. These results establish a reproducible testbed and a bounded proof of concept, not evidence of live-LLM performance.

1 Introduction

Modern agentic applications combine language models with retrieval, code execution, external APIs, specialist agents, verification, and recovery logic. Before any component can help, a controller must choose whether to invoke it. This meta-decision affects accuracy, cost, latency, and exposure to component failure. A direct answer is efficient but may lack evidence or computation; indiscriminate tool use adds overhead; and a fixed workflow can fail when one component is unavailable.

Prior research demonstrates the value of adaptive retrieval (Jeong et al. 2024), cost-aware model routing (Chen, Zaharia, and Zou 2024; Ong et al. 2025), optimized function-call plans (Kim et al. 2024), and reflective planning (Yao et al. 2023; Zhang et al. 2026). Agent benchmarks evaluate planning and tool execution (Liu et al. 2024; Yao et al. 2024), but fewer studies isolate the policy that selects among *different classes* of cognitive and execution operations. This leaves two questions entangled: whether a controller identifies relevant capabilities and whether composing several capabilities is better than selecting one.

We study that layer with an executable benchmark. Each task is presented to a policy only as raw text. A route may contain decomposition, retrieval, code execution, delegation, and verification before answering. Unlike our preliminary offline simulator, success is not sampled from annotated needs: operations transform task state, produce a candidate answer, and encounter deterministic failure conditions; an exact evaluator then checks the result. Labels are used only to train operation predictors and to analyze route quality.

Our contributions are:

- an executable meta-routing benchmark with machine-checked data-analysis, research, and document-processing tasks, held-out templates, a locked lexical-shift challenge split, and component-outage cases;
- a calibrated raw-text router that predicts multiple operation types and composes them under explicit action-count and cost budgets;
- comparisons with direct, random, keyword, static-workload, fixed-agent, one-shot learned, and oracle policies on success, cost, local latency, budget compliance, and route quality; and
- paired statistical analysis and ablations separating composition, character-level generalization, calibration, and budget enforcement.

The central result is deliberately scoped. The learned router solves all 108 standard test tasks while using 1.76 normalized cost units, compared with 93.5% success and 3.08 units for a strong static policy. Under lexical shift, however, it misses aggregate computation, multi-hop retrieval, and conflict verification, reaching only 75.9%. The result shows both the value of operation composition and the fragility of lightweight text routing.

2 Related Work

Model and retrieval routing. FrugalGPT constructs model cascades to reduce inference cost, while RouteLLM

arXiv submission. This version includes author names and affiliations.

learns to select between stronger and weaker models from preference data (Chen, Zaharia, and Zou 2024; Ong et al. 2025). CP-Router uses uncertainty to choose between standard and long-reasoning models (Su et al. 2026); ICL-Router represents endpoint capabilities through in-context vectors (Wang et al. 2026); and ZeroRouter uses a model-agnostic latent space to onboard unseen endpoints while optimizing accuracy, cost, and latency (Yan et al. 2026). Adaptive-RAG predicts question complexity and selects no retrieval, single-step retrieval, or iterative retrieval (Jeong et al. 2024). These methods establish conditional routing as an operational optimization problem. Our decision variable is different: we compose heterogeneous operations around a model rather than choose a model or retrieval depth.

Tool learning and workflow construction. Toolformer learns when and how to call APIs (Schick et al. 2023). API-Bank and ToolLLM provide resources for API planning, retrieval, and execution (Li et al. 2023; Qin et al. 2024); AnyTool combines hierarchical retrieval with reflection after failed calls (Du, Wei, and Zhang 2024); and RESTful-Llama studies an industry natural-language-to-API pipeline (Xu et al. 2024). LLMCompiler exposes function dependencies for parallel execution and lower latency (Kim et al. 2024), while Smurfs distributes tool planning across specialized agents and reduces context overhead (Chen, Liang, and Wang 2025). These systems optimize tool-centered plans. We treat tool use as one optional operation beside decomposition, code, delegation, verification, and direct answering.

Planning and verification. ReAct interleaves reasoning with environmental action (Yao et al. 2023); SPIRAL separates planning, simulation, and critique inside reflective tree search (Zhang et al. 2026). ACPBench generates tasks with provably correct solutions from formal planning domains and finds uneven planning abilities across models (Kokel et al. 2025). Our method is simpler: it predicts one bounded route before execution and does not perform search or observation-conditioned replanning. This simplicity makes the effect of route composition easier to isolate, at the cost of weaker adaptivity.

Agent evaluation and diagnosis. AgentBench, SWE-bench, and τ-bench evaluate interactive, executable, or user-mediated behavior (Liu et al. 2024; Jimenez et al. 2024; Yao et al. 2024). PlanningArena evaluates planning and tool selection at macro and micro levels (Zheng et al. 2025); ToolSandbox adds stateful execution, intermediate milestones, and conversational evaluation (Lu et al. 2025). AgentDiagnose measures trajectory competencies including decomposition, observation reading, verification, and backtracking (Ou et al. 2025). Our benchmark is smaller and less realistic, but provides controlled supervision and paired policy comparisons over several operation classes.

Novelty boundary. The contribution is not a new foundation model or general-purpose planner. It joins ideas normally evaluated separately: task-conditioned routing, multi-operation workflow construction, cost constraints, executable grading, and trajectory-level diagnosis. The matched one-shot learner sees the same text representation and supervision as the compositional router, so their difference isolates route composition more directly than comparisons against direct prompting alone.

3 Problem Formulation

Let a task be raw text x with cost budget B. A route $r = (a_1, \ldots, a_m, A)$ terminates in answer action A and draws support operations from

$$\mathcal{O} = \{D, T, C, G, V\}, \tag{1}$$

where D is decomposition, T retrieval/tool use, C code execution, G specialist delegation, and V verification. Each operation has preregistered normalized cost $c(a)$. A route is feasible when $m \leq M$ and $\sum_{a \in r} c(a) \leq B$.

Executing r on task x produces candidate answer $\hat{y}$ and trace z. Success is $S(x, r) = 1$ when $\hat{y} = y$ and 0 otherwise under a task-specific exact evaluator. The policy objective is to maximize success under route constraints, while raw success, cost, latency, and failure type are reported separately. Operation labels $L(x) \subseteq \mathcal{O}$ identify a minimum valid route for training and route-quality analysis; the executor never converts label overlap directly into success.

4 Executable Benchmark

Tasks and Operations

The suite contains 504 tasks: 216 train, 72 development, 108 test, and 108 challenge examples, each balanced across three workloads. Prompt templates are disjoint across splits. The challenge prompts were locked after observing standard-test development behavior and use more distant paraphrases; no model or threshold was changed after their first run. Values, record identifiers, evidence entries, and document fields are generated deterministically from a fixed seed. Table 1 summarizes the executable families.

Operations manipulate an explicit state rather than receiving success credit from route labels. For example, retrieval over conflicting records initially returns the older entry; verification selects the record with the latest year. Invoice extraction initially exposes a stated but incorrect total; code computes the item sum and verification selects the recomputed value. In multi-hop research, retrieval without decomposition returns only the intermediate laboratory. Retrieval-outage tasks mark T unavailable and provide the answer only through G. These mechanisms yield interpretable incorrect-answer, no-answer, and unavailable-component failures.

Execution is order sensitive. Decomposition writes task-specific subgoals or field requirements into state; retrieval and code consume those artifacts; verification can revise a candidate only when the prerequisite evidence or computation exists. If an unavailable operation is invoked, execution stops with a typed component failure. Otherwise, answering emits the current candidate or applies a direct-answer parser for literal tasks. Thus an irrelevant extra operation may add cost without changing the answer, while a missing or misordered prerequisite changes the actual artifact presented to the evaluator.

Table 1: Executable task families. Direct literal tasks occur in every workload. Minimum routes exclude the final answer action.

Workload	Task families	Minimum route	Machine-checked behavior
Data analysis	Aggregate; filtered sum	C; D, C	Compute numeric aggregates; applying code before decomposition sums the wrong subset
Research	Fact; multi-hop; conflicting sources; outage	T; D, T; T, V; G	Retrieve frozen records, follow evidence chains, select newest evidence, or delegate when retrieval is unavailable
Document processing	Field extraction; invoice reconciliation; locale date; cross-field identifier	T; T, C, V; T, G; D, T, V	Parse fields, recompute incorrect totals, normalize dates, and combine validated fields
All	Direct literal	none	Return an explicitly supplied literal without support operations

The benchmark design separates four sources of difficulty. *Selection* tasks require one non-answer operation. *Composition* tasks require two or three dependent operations. *Conflict* tasks provide a plausible but wrong intermediate candidate that verification must correct. *Availability* tasks invalidate the usual retrieval path and require delegation. This taxonomy supports failure analysis beyond aggregate accuracy and prevents a direct policy from succeeding through answer-format heuristics on nonliteral tasks.

The operation costs are .45 for decomposition, 1.00 for retrieval, 1.15 for code, 1.40 for delegation, .65 for verification, and .30 for answering. Every task uses budget $B = 4.5$. Costs represent relative component usage, not dollars or tokens.

Raw-Text Operation Model

The router receives only the prompt text. We construct binary word unigrams, word bigrams, and character 3–5-gram features $\phi(x)$. For each operation o, a regularized logistic head estimates

$$p_o(x) = \sigma(w_o^\top \phi(x) + b_o). \tag{2}$$

Heads minimize class-balanced binary cross entropy with ℓ_2 regularization on the training split. Optimization uses 500 full-batch gradient steps with a decaying learning rate. We temperature-scale each head on development data by minimizing Brier score over a fixed temperature grid. Development Brier scores range from .00004 for delegation to .102 for retrieval.

For operation o, the training objective is

$$\mathcal{L}_o = \mathrm{WBCE}(y, p_o(x)) + \lambda \|w_o\|_2^2, \tag{3}$$

where weighted binary cross entropy (WBCE) gives positive and negative examples equal total weight. We use learning rate $.35/\sqrt{1 + e/50}$ at epoch e, $\lambda = .002$, and minimum document frequency 2. The resulting representation has 3,349 features. Training all five heads takes approximately .12 seconds on the reported CPU. Temperature selection chooses .5 for every head; the no-calibration ablation below tests whether this affects routes.

The route threshold τ is selected from $\{.30, .35, \ldots, .70\}$ on development labels, prioritizing exact-route accuracy, then action F1, then lower cost. This yields $\tau = .40$. Candidate operations with $p_o(x) \geq \tau$ are ranked by $(p_o - \tau)/c(o)$ and greedily added while respecting $M = 3$ and B. Operations are placed in canonical order D, T, C, G, V, followed by A. Algorithm 1 gives the procedure.

Algorithm 1 Budget-Aware Text Meta-Routing

Require: text x, probabilities p_o, threshold τ, budget B
$Q \leftarrow \{o : p_o(x) \geq \tau\}$
sort Q by $(p_o(x) - \tau)/c(o)$ descending
$R \leftarrow [\,]$; $k \leftarrow c(A)$
for o in Q **do**
  if $|R| < 3$ and $k + c(o) \leq B$ **then**
    append o to R; $k \leftarrow k + c(o)$
  end if
end for
reorder R as D, T, C, G, V
return R followed by A

Baselines

We compare seven alternatives. *Direct* always answers. *Random* selects zero to three operations deterministically from the task identifier. *Keyword* uses raw-text rules. *Static workload* uses one route per workload. *Fixed agent* always executes all five support operations. *Learned one-shot* uses the same calibrated model but selects at most its highest-probability operation. *Oracle* executes the annotated minimum route and serves as an upper bound. Static and fixed policies are deliberately strong: they solve all tasks except research outages.

5 Experimental Protocol

We fit on train, calibrate and select τ on development, and report both standard test and the subsequently locked challenge split. No standard-test or challenge labels are used for fitting or threshold selection. Every policy runs every task in each evaluation split. We report machine-checked success, 2,000-sample task-bootstrap 95% intervals, normalized route cost, local wall-clock route-plus-execution latency, budget compliance, exact-route match, and micro action precision/recall/F1. Paired success differences use task bootstrap intervals and an exact two-sided sign test over discordant outcomes.

Table 2: Held-out executable results ($n = 108$ per policy). CI is the bootstrap interval for success. Latency is local milliseconds.

Policy	Success	95% CI	Cost	F1
Oracle	1.000	[1.000,1.000]	1.50	1.000
Learned budget	**1.000**	[1.000,1.000]	1.76	.910
Static workload	.935	[.880,.972]	3.08	.535
Fixed agent	.935	[.880,.972]	4.95	.414
Keyword	.583	[.491,.676]	1.13	.810
Learned one-shot	.565	[.472,.657]	1.36	.643
Random	.407	[.315,.500]	1.68	.303
Direct	.259	[.185,.352]	.30	.000

Table 3: Results on 108 locked lexical-shift challenge tasks.

Policy	Success	Cost	Action F1
Oracle	1.000	1.50	1.000
Static workload	.935	3.08	.535
Fixed agent	.935	4.95	.414
Learned budget	.759	1.56	.810
Learned one-shot	.417	1.19	.549
Random	.380	1.87	.304
Keyword	.324	.54	.331
Direct	.259	.30	.000

Experiments ran CPU-only on an Apple M5 system with 10 cores and 16 GB memory under macOS, Python 3.12.13, and NumPy 2.5.0. Routing and execution are repeated five times per task solely to stabilize sub-millisecond timing, then averaged. The local operations do not call an LLM or network service; latency therefore measures implementation overhead, not deployment latency.

6 Results

Main Comparison

The learned budget policy solves all 108 tasks. Relative to static workload routing, it improves success by 6.48 points (paired 95% CI [2.78,11.11], exact sign $p = .0156$) and reduces mean cost by 1.32 units, or 43.0%. The difference consists of seven retrieval-outage tasks solved only by learned delegation. Relative to learned one-shot routing, composition improves success by 43.52 points ([34.26,52.78], $p < 10^{-13}$) for .40 additional cost. This is the strongest evidence that multiple predicted operations, not text supervision alone, drive task completion.

The learned policy has 100% budget compliance versus 66.7% for static routing and 0% for the fixed agent. Its mean local latency is .17 ms, compared with less than .01 ms for rule policies; this difference is measurable but operationally negligible relative to model or tool calls.

Locked Challenge Split

The challenge reverses the comparison with static routing. Learned success is 17.59 points lower (paired 95% CI [-27.78,-7.41], $p = .0013$), although cost is 1.52 units, or 49.4%, lower. The learned policy still exceeds one-shot routing by 34.26 points ([25.93,44.44], $p < 10^{-10}$). All 26 learned failures cluster in three families: 12 aggregate tasks, seven multi-hop research tasks, and seven conflicting-source tasks. Thus the executor and route composition remain effective when the required operations are recalled; the failure is operation prediction under paraphrase. Figure 1 contrasts the two splits.

Table 4: Learned-router ablations on the same test tasks.

Condition	Success	Cost	Action F1
Full	**1.000**	1.76	.910
No calibration	1.000	1.76	.910
No budget enforcement	1.000	1.76	.910
Word features only	.870	1.37	**.948**
Single operation	.565	1.36	.643

Table 5: Threshold sensitivity without refitting. Costs are mean normalized route costs.

τ	Test	Cost	Challenge	Cost
.30	1.000	1.76	.759	1.56
.40	1.000	1.76	.759	1.56
.50	1.000	1.76	.722	1.52
.60	1.000	1.50	.694	1.50
.65	.963	1.48	.639	1.17
.70	.926	1.45	.546	1.09

Route Quality and Ablations

Although learned and oracle success are equal, their routes differ. The learned policy has .741 exact-route match, .834 action precision, 1.000 recall, and .910 F1. It invokes retrieval on 77 tasks versus 49 for the oracle, explaining its .26 higher cost. Extra operations do not change answers in the current executor, but they expose an efficiency gap hidden by success.

Removing character features lowers success by 12.96 points despite increasing action F1. The word-only model is more precise but misses operations under held-out phrasing; character features favor recall. Restricting the model to one operation produces the largest loss. Calibration and budget ablations are identical to the full policy because they do not alter threshold crossings or activate the budget boundary on this split. They should not be credited for the main gain under the tested regime.

Sensitivity and Failure Analysis

Table 5 shows a stable region from $\tau = .30$ to $.45$. Larger thresholds trade cost for recall and harm the challenge split first. At $\tau = .60$, standard success remains perfect and routes match the oracle exactly, but challenge success is only 69.4%. Choosing a threshold solely from the standard distribution therefore understates generalization risk.

The challenge failures are sharply attributable. On aggregate prompts, mean code probability is below threshold (a representative task has $p_C = .192$). Multi-hop prompts retain retrieval ($p_T = .882$) but miss decomposition ($p_D = .051$), returning the intermediate laboratory. Conflicting-source

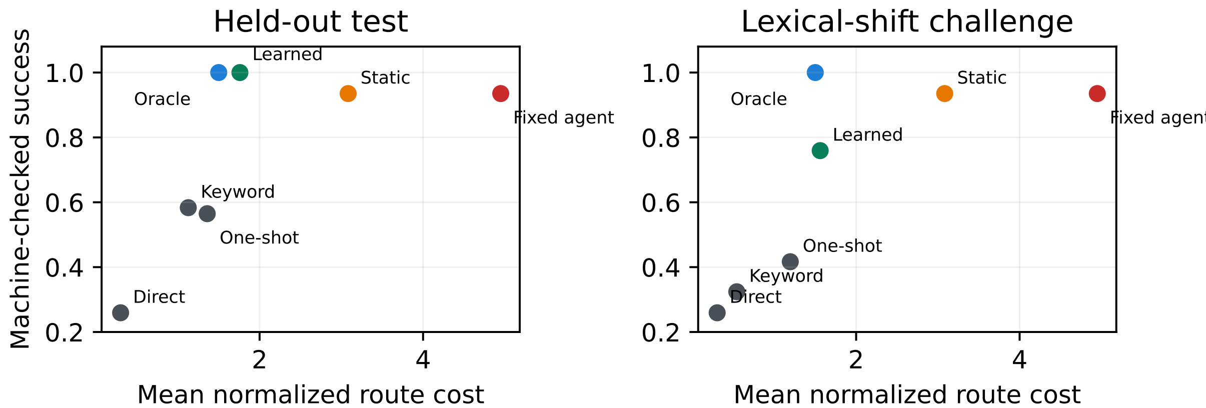

Figure 1: Machine-checked success versus route cost. The learned router matches oracle success on the standard test but loses to static routing under lexical shift, while remaining substantially cheaper.

prompts retain retrieval ($p_T = .985$) but miss verification ($p_V = .009$), returning stale evidence. In contrast, the outage paraphrase still selects delegation ($p_G = .675$), and invoice reconciliation retains retrieval, code, and verification probabilities above .67. These traces show that failure is not uniform semantic collapse; it is operation-specific recall under paraphrase.

We also refit with 36, 72, 108, 144, and 216 training tasks. Standard and challenge success are unchanged at every size. Because each family repeats one training template with different values, this flat curve indicates redundancy, not exceptional sample efficiency. Broader template and domain diversity is necessary before drawing scaling conclusions.

7 Discussion

Interpretation and Novelty

Three conclusions follow within the benchmark. First, static workflows remain strong when workload identity predicts the needed route; they waste cost on easy tasks and fail under component outages, but are more robust to paraphrase than the learned lexical router. Second, predicting a single operation is insufficient for tasks such as filtered computation, multi-hop retrieval, invoice reconciliation, and locale normalization on both evaluation splits. Third, standard-test success alone overstates router quality: the learned policy matches the oracle on answers while selecting 28 unnecessary retrieval operations and then loses 24.1 success points under lexical shift.

The method inherits task conditioning from adaptive retrieval, cost awareness from model routing, and modular operations from agent planning. Its novel unit of control is the heterogeneous operation route. Endpoint routers decide which model should answer; retrieval routers decide how much evidence to acquire; tool planners arrange calls within a tool-centered workflow. Our policy decides which kinds of reasoning and execution should constitute the workflow before those lower-level mechanisms run. The executable suite and matched one-shot control make this distinction empirically testable.

Deployment Implications

A practical system could use this router above existing model and tool routers. A low-cost gate predicts whether direct answering is sufficient; a multi-label policy then proposes support operations under cost and latency budgets; endpoint selection occurs inside each operation. Typed traces support per-component reliability estimates, budget audits, and fallback rules. The outage cases illustrate why route choice should depend on component availability rather than only task category.

The present classifier is intentionally lightweight and interpretable. A deployment router should replace lexical features with representations trained across broader tasks, condition on execution history and component health, and replan after observations. A constrained contextual bandit or Markov decision process could optimize success and cost online, while retaining static routes as fallbacks when uncertainty is high.

The challenge result suggests a concrete hybrid. When calibrated operation probabilities are diffuse or fall near threshold, the controller can back off to a workload route rather than answer with an incomplete composition. On the current challenge set, static routing covers the aggregate, multi-hop, and conflict families missed by the learner, while learned delegation covers the static policy's outage failures. A learned confidence gate between these policies is a natural next baseline, but we leave it unevaluated to avoid tuning on the locked challenge split.

Limitations and Threats to Validity

Construct validity. Machine-checked answers are stronger than sampled success, but the operations are deterministic Python components rather than LLMs, sandboxes, or network tools. Normalized route costs are design constants, not tokens or dollars. Local sub-millisecond latency should not be extrapolated to deployed agents.

Internal validity. Minimum-route labels and executor semantics are designed together. The executor does not directly score label overlap, but task families still encode the authors' assumptions about which operations should help. Perfect learned success reflects complete recall on a finite templated suite, not general intelligence. The threshold and temperatures are selected on only 72 development tasks.

External validity. The locked challenge split exposes surface-form fragility but does not test new domains, languages, tools, or long interactive horizons. PlanningArena and ToolSandbox contain richer stateful interactions (Zheng et al. 2025; Lu et al. 2025); SWE-bench and τ-bench provide more realistic executable and user-mediated outcomes (Jimenez et al. 2024; Yao et al. 2024). Evaluation on those settings, semantic encoders, multiple model families, and adversarial distribution shifts is required.

Benchmark-development validity. The standard test influenced benchmark and feature development before the challenge split was introduced. We therefore treat standard-test performance as an in-benchmark result, not a pristine confirmatory estimate. The challenge templates were subsequently locked and the first-run outcome was retained without model changes, but they were still authored with knowledge of the benchmark ontology. An independently constructed task suite is needed for confirmatory evaluation.

Statistical validity. Task bootstrap intervals quantify variation over this finite suite. The 100% success interval is degenerate and does not estimate performance on an external task population. Exact sign tests cover paired outcome differences but do not correct across every exploratory comparison. Future work should preregister a larger benchmark and report family-stratified uncertainty.

Ethical Statement

The benchmark contains synthetic records, documents, and numerical tasks; it uses no personal, proprietary, defense, or human-subject data and performs no external actions. Deployed meta-routing can nevertheless create privacy, security, and accountability risks when data are sent to external services, code is executed, or decisions are delegated. Practical systems should use sandboxing, allowlisted tools, data minimization, hard budgets, audit logs, and human escalation for high-impact tasks. Benchmark success must not be interpreted as evidence of safety or suitability for autonomous consequential decisions.

References

Chen, J.; Liang, J.; and Wang, B. 2025. Smurfs: Multi-Agent System using Context-Efficient DFSDT for Tool Planning. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 3281–3298. Association for Computational Linguistics.

Chen, L.; Zaharia, M.; and Zou, J. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*.

Du, Y.; Wei, F.; and Zhang, H. 2024. AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls. In *Proceedings of the 41st International Conference on Machine Learning*.

Jeong, S.; Baek, J.; Cho, S.; Hwang, S. J.; and Park, J. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 7036–7050. Association for Computational Linguistics.

Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations*.

Kim, S.; Moon, S.; Tabrizi, R.; Lee, N.; Mahoney, M. W.; Keutzer, K.; and Gholami, A. 2024. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 24370–24391.

Kokel, H.; Katz, M.; Srinivas, K.; and Sohrabi, S. 2025. ACP-Bench: Reasoning About Action, Change, and Planning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(25): 26559–26568.

Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116. Association for Computational Linguistics.

Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; Yang, K.; et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*.

Lu, J.; Holleis, T.; Zhang, Y.; Aumayer, B.; Nan, F.; Bai, H.; Ma, S.; Ma, S.; Li, M.; Yin, G.; Wang, Z.; and Pang, R. 2025. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 1160–1183. Association for Computational Linguistics.

Ong, I.; Almahairi, A.; Wu, V.; Chiang, W.-L.; Wu, T.; Gonzalez, J. E.; Kadous, M. W.; and Stoica, I. 2025. RouteLLM: Learning to Route LLMs with Preference Data. In *International Conference on Learning Representations*.

Ou, T.; Guo, W.; Gandhi, A.; Neubig, G.; and Yue, X. 2025. AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 207–215. Association for Computational Linguistics.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*.

Schick, T.; Dwivedi-Yu, J.; Dessi, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, volume 36.

Su, J.; Lin, F.; Feng, Z.; Zheng, H.; Wang, T.; Xiao, Z.; Zhao, X.; Liu, Z.; Cheng, L.; and Wang, H. 2026. CP-Router: An Uncertainty-Aware Router Between LLM and LRM. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(39): 33065–33073.

Wang, C.; Li, H.; Zhang, Y.; Chen, L.; Chen, J.; Jian, P.; Zhang, Q.; and Hu, S. 2026. ICL-Router: In-Context Learned Model Representations for LLM Routing. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(39): 33413–33421.

Xu, H.; Zhao, R.; Wang, J.; and Chen, H. 2024. RESTful-Llama: Connecting User Queries to RESTful APIs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 1433–1443. Association for Computational Linguistics.

Yan, C.; Zhang, W.; Ning, Z.; Xu, F.; Tao, Z.; Zhang, L.; Yin, B.; and Zhang, Y. 2026. Breaking Model Lock-in: Cost-Efficient Zero-Shot LLM Routing via a Universal Latent Space. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(43): 36483–36490.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Zhang, Y.; Ganapavarapu, G.; Jayaraman, S.; Agrawal, B.; Patel, D.; and Fokoue, A. 2026. SPIRAL: Symbolic LLM Planning via Grounded and Reflective Search. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(43): 36527–36535.

Zheng, Z.; Cui, T.; Xie, C.; Pan, J.; Chen, Q.; and He, L. 2025. PlanningArena: A Modular Benchmark for Multidimensional Evaluation of Planning and Tool Learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 31047–31086. Association for Computational Linguistics.